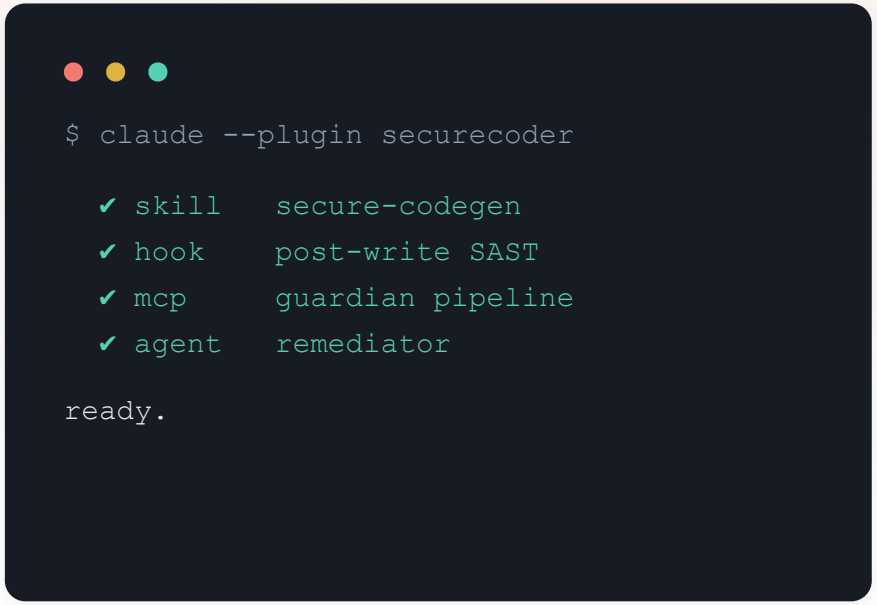
CLAUDE CODE PLUGIN · HOW IT WORKS

SecureCoder

Security-aware code generation,
inside the agent loop.

STG Group at TU Darmstadt · July 2026



/plugin install securecoder

THE PROBLEM

LLMs still write vulnerable code

40%

of AI-generated programs contained exploitable vulnerabilities.
Pearce et al., IEEE S&P 2022

The same old bugs

SQL injection, path traversal, hardcoded secrets. The CWE top list has barely changed.

Agents make it worse

An agent writes and runs code far faster than anyone can review it.

user_service.py

```
@app.route("/user")
def get_user():
    uid = request.args["id"]
    q = f"SELECT * FROM users
         WHERE id={uid}"
    return db.execute(q)
```

CWE-89 · SQL injection
Written in seconds, exploitable from day one.

THE SOLUTION

Put the checks inside the loop

Guide the generation

Enrich every prompt with security constraints, so code is born secure instead of patched later.

Close the feedback loop

Agents can correct their own mistakes, when analyzer findings flow straight back into their context.

One channel to the analyzers

Classical, deterministic analysis and agentic reasoning talk through a single, unified interface.

Verify until clean

Fix and re-check iteratively, with clear stopping rules; security is proven, not assumed.

DIVISION OF LABOR

Claude runs the loop, the server runs the checks

Claude handles generation, triage and fixes; the plugin's MCP server delivers fast, deterministic security analysis.

PIPELINE ROLE		IMPLEMENTED BY
`Generate → analyze → fix loop`	→	**Claude Code's agentic loop**
`Security constraint injection`	→	**secure-codegen skill + MCP enricher**
`Guardians: CodeQL, semgrep, bandit`	→	**Zero-dependency MCP server**
`Triage & retry decisions`	→	**Claude in-context; the server keeps score**

THE PLUGIN

One install, three lines of defense

1 · BEFORE

The secure-codegen skill enriches the prompt with contract-preserving constraints via MCP.

2 · AFTER EVERY WRITE

A PostToolUse hook runs syntax, semgrep and bandit checks and puts findings in context.

3 · UNTIL CLEAN

The remediator agent fixes and re-analyzes, one session id, until the verdict is clean.

`securecoder-analysis MCP server` runs the analysis for all three stages.

UNDER THE HOOD

Two speeds of analysis

Fast lane · seconds

Runs after every file write.

`syntax check` · `semgrep` · `bandit`

Deep lane · on demand

Used for the first pass and the final check.

`CodeQL · security-extended suites`

The server tracks each session: which findings were fixed, which are new, which persist, and when to stop retrying.

IN ACTION

What Claude sees

hook output

```
[securecoder] 2 findings

HIGH  CWE-89  SQL injection
  → use parameterized queries

MED   CWE-798 hardcoded secret
  → move to a secret manager
```

remediation report

```
attempt     2 / 7
resolved    2
persisting  0

deep pass   CodeQL ✔

verdict     CLEAN ✔
```

All of this happens on its own during a normal coding session.

FEATURES

Built for real-world repos

Feedback, not noise

Per-rule remediation guidance with code snippets, deduplicated before it reaches Claude.

Knows when to stop

Tracks resolved, new and persisting findings per session; soft and hard limits end unproductive fix loops.

Configurable per repo

.securecoder.json controls analyzers, custom CodeQL query packs, exclude patterns and retry limits.

Degrades gracefully

Analyzers are picked up from PATH; anything missing is skipped and never breaks the workflow.

Security as a first-class constraint of the agent loop.

Checked before generation, after every write, and until the analyzers come back clean.

Node ≥ 18 · zero npm dependencies

/plugin install securecoder

skill · hook · mcp server · agent

Thank you · SecureCoder plugin for Claude Code